Course 2 Lab: AI Icebreaker Bot with LlamaIndex

Course 2 Module 3 Lab: Build an AI Icebreaker Bot (LlamaIndex & IBM Granite)

This is your distilled, "No-Scrolling" cheat sheet for the final Course 2 lab.

[SKIP] The SKIP Pile (Boilerplate / Busywork)

- Part 7 (CLI Application): Terminal input loop (while True: prompt = input()).
- Part 8 (Gradio Interface): Wrapping the query engine inside gr.Interface(fn=icebreaker_bot, inputs="text", outputs="text").launch().
- Why skip? As established, Gradio is only used for Coursera demos. In production, we serve LLMs via FastAPI.

[SKIM] The SKIM Pile (Read & Understand)

- Part 1 & 3 (Configuration & Mock Data):
- Loading a text file representing a person's LinkedIn profile (e.g. linkedin_profile.txt).
- Part 5 (IBM Granite Models in LlamaIndex):
- Defining WatsonxLLM and WatsonxEmbeddings to configure LlamaIndex's global settings:
python
from llama_index.core import Settings
Settings.llm = watsonx_llm
Settings.embed_model = watsonx_embed

[MUST KNOW] The STRUGGLE Pile (Core Interview Knowledge)

1. The Icebreaker RAG Concept

- The Problem: Given a raw text export of someone's resume or LinkedIn profile, extract interesting icebreakers, topic starters, or summary points for a meeting.
- The Solution: Load the profile text into LlamaIndex -> Build a VectorStoreIndex -> Create a QueryEngine with a custom Icebreaker prompt!

2. Part 4 & 6: LlamaIndex Pipeline Implementation

python
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex, PromptTemplate

1. Load Profile Document

documents = SimpleDirectoryReader(input_files=["linkedin_profile.txt"]).load_data()

2. Build Vector Index (Handles embedding + indexing automatically)

index = VectorStoreIndex.from_documents(documents)

3. Custom Icebreaker Prompt Template

icebreaker_prompt = PromptTemplate(
"Given the profile context below, extract 2 interesting facts and 1 icebreaker conversation starter.\n"
"Context: {context_str}\n"

```
"Question: {query_str}\n"
"Answer:"
)
```

4. Create Query Engine with Custom Prompt

```
query_engine = index.as_query_engine(
text_qa_template=icebreaker_prompt,
similarity_top_k=3
)
```

5. Execute Query

```
response = query_engine.query("Generate icebreakers for this candidate.")
print(response)
```

Architecture Summary

LinkedIn Profile (.txt) -> SimpleDirectoryReader -> VectorStoreIndex -> Custom Icebreaker Prompt -> QueryEngine -> Instant Meeting Icebreakers